

CINE MATCH

Sommerprojekt 2025 · Clemens Altrichter · 3BHITM

"Nie wieder endlos scrollen — einfach swipen, abstimmen, schauen."

INHALTSVERZEICHNIS

01 Projektidee & USPs

02 Datenbankstruktur

03 PHP Code-Highlights

04 Live-Demo

Das Problem

- **Filmabend: was schauen?**
 - 30 Minuten Diskussion, kein Ergebnis
 - Jeder scrollt auf seinem eigenen Netflix
- Kein gemeinsamer Entscheidungsprozess
- Niemand weiß was die anderen wollen

Die Lösung:

- Universell anwendbare webapp
- Watch Party mit Freunden starten
- Gemeinsam durch alle Vorschläge swipen
- Film mit den meisten Likes gewinnt

Was CineMatch einzigartig macht



Eigene Listen

Nur Filme die die Gruppe wirklich sehen will



Demokratisch

Jeder hat gleich viel Einfluss — niemand setzt sich durch



Schnell

Kein endloses Scrollen — Swipe entscheidet in Sekunden



Plattformfrei

Egal ob Netflix, Disney+, Kino — app-unabhängig

10 Tabellen

BASISTABELLEN

user

userID username email avatar

movie

movieID tmdbID title poster

list

listID userID name

list_movie

listID movieID

notification

id fromID toID status partyID

friend

userID1 userID2 status

Temporäre Daten „dauerhaft“ speichern

WATCHPARTY FLOW

watchparty

watchpartyID userID name status date chosenMovieID

partymember

watchpartyID userID status

partylist

watchpartyID listID

swipes

userID movieID watchpartyID liked

Session-Schutz auf allen Endpunkten

Warum:

- Alle scrape*.php Dateien geben Daten zurück — ohne Schutz für jeden abrufbar
- header() Redirect funktioniert nicht bei JS-Fetches → JSON mit Code 401 statt HTML
- JS-Seite fängt 401 ab und leitet weiter — safeFetch() als zentrale Hilfsfunktion

```
// PHP — jede scrape*.php
if (!isset($_SESSION['userID'])) {

    header('Content-Type: application/json');

    echo json_encode(["code" => 401]);

    exit;
}

// JS — auth.js
async function safeFetch(url, opts = {}) {

    const data = await (await fetch(url, opts)).json();

    if (data.code === 401) {
        location.href =
            '../userSys/index.html';

        return null;
    }

    return data;
}
```

Swipe-Logik & Ergebnis-Query

Swipe speichern

```
INSERT INTO swipes
    (userID, movieID, watchpartyID, liked)
VALUES (?, ?, ?, ?)
ON DUPLICATE KEY UPDATE
    liked = ?
```

- ON DUPLICATE KEY → kein doppelter Swipe möglich
- liked als boolean (0/1) gespeichert

Gewinner ermitteln

```
SELECT m.title, COUNT(*) AS likes
FROM swipes s
JOIN movie m ON m.movieID = s.movieID
WHERE s.watchpartyID = ? AND s.liked = 1
GROUP BY m.movieID
ORDER BY likes DESC
```

- COUNT(*) zählt alle Likes pro Film
- ORDER BY DESC → meistgelikter Film zuerst

Echtzeit ohne WebSockets — Polling

- **Kein WebSocket-Server nötig**
 - PHP unterstützt keine persistenten Verbindungen
- Lösung: Browser fragt alle 3 Sekunden an
 - `setInterval(() => pollStatus(), 3000)`
- Polling ist pur lese basiert, anders als bei anderen varianten wo die schnittstelle die informationen selber sendet
- Status-Felder in der watchparty Tabelle
 - open → Warteraum
 - finished → Ergebnis
- Alle Clients merken Statuswechsel gleichzeitig

```
// JS - watchpartyWait.js
let pollInterval;

function pollStatus() {

  safeFetch(`scrapeWatchpartys.php
    ?getPartyStatus=${
      partyID}`)
    .
    then(data => {

      renderMembers(data.members);

      if (data.status === 'active') {

        clearInterval(pollInterval);
        location.href =
          `watchpartySwipe.html
            ?partyID=${
              partyID}`;
      }
    });
}
```

```
pollInterval = setInterval
```


TMDB API — Filmdaten automatisch laden

Wie es funktioniert:

- **TMDB bietet eine kostenlose REST-API**
 - Poster, Titel, Beschreibung, Bewertung — alles automatisch
 - Kein manuelles Eintragen von Filmdaten nötig
- **loadMoviesIntoDB.php** lädt einmalig 500+ Filme
 - Speichert nur relevante Felder in die movie-Tabelle
- **Auf der Browse-Seite:** Suche läuft gegen lokale DB
 - Kein API-Call bei jeder Suchanfrage — schnell & offline-fähig

```
// loadMoviesIntoDB.php
$url = "https://api.themoviedb.org/3/"

    . "movie/popular?api_key=" . API_KEY;

$response = file_get_contents($url);

$data = json_decode($response, true);

foreach ($data['results'] as $movie) {

    $stmt = $conn->prepare(
        "INSERT IGNORE INTO
        movie (tmdbID,title,overview,
            voteAVG,poster) VALUES(?,?,?,?,?)"
    );

    $stmt->bind_param("issds",
        $movie['id'], $movie['title'],
        ...
    );

    $stmt->execute();
}
```

Notifications & Friend Requests

Eine Tabelle — zwei Verwendungen:

- notification speichert Invites UND Friend Requests
 - content = 'invite' → Watch Party Einladung
 - content = 'friendrequest' → Freundschaftsanfrage
- Status-Felder steuern den Flow
 - pending → noch nicht beantwortet
 - accepted → angenommen
 - rejected → abgelehnt
- Invite annehmen → 2 DB-Updates gleichzeitig
 - partymember.status = 'joined'
 - notification.status = 'accepted'
- Friend Request annehmen → friend.status = 'accepted'

```
// Invite senden (watchpartyConf.php)
$stmt = $conn->prepare(
    "INSERT INTO
    notification

    (fromID,toID,content,status,partyID)

    VALUES (?,?,'invite','pending',?)"
);

// Invite annehmen (scrapeWatchpartys.php)
if (isset($_GET['acceptInvite'])) {

    // partymember updaten

    $conn->prepare("UPDATE partymember

    SET status='joined'

    WHERE watchpartyID=? AND userID=?");

    // notification updaten

    $conn->prepare("UPDATE notification

    SET status='accepted'

    WHERE partyID=? AND toID=?");
}
```

LIVE DEMO

1

Registrieren & Anmelden

Neuen Account erstellen, Session zeigen

2

Listen & Filme

Watchlist öffnen, Film per Suche hinzufügen

3

Watch Party erstellen

Party konfigurieren, Friend einladen

4

Invite annehmen

Auf zweitem Account Invite annehmen, Warteraum

5

Swipen & Ergebnis

Beide Accounts swipen, Gewinner anzeigen

DANKE

für eure Aufmerksamkeit

9 Datenbanktabellen · PHP Sessions & Prepared Statements · TMDB API Integration · Realtime Polling · Mobile-First Design

Fragen?